

Addendum to Final In-Class Test – SDBP 031

Assigned: 2026-09-18

Due: 2026-09-30 before 11.59PM

Total Marks: 10

Submission Procedure:

- Package your screenshots and text file deliverables into a single ZIP file.
- Create a **ZIP** file of your solution and upload it to **BlackBoard**.
 - o Name your ZIP file following the format below
 - E.g. SDBP031-Addendum-Joe-Smith-N#####.zip
- **DO NOT EMAIL YOUR ASSIGNMENT** – Assignments submitted by email will not be accepted or evaluated.

Contents

Part	Contents
The assignment at a glance	What you're building · What you'll have learned · The one idea behind the setup
Step by step	Step 1 · Source Control (15 min) · Step 2 · API requirements (15 min) · Step 3 · Provision the cloud database (20 min) · Step 4 · App Service configuration (30 min) · Step 5 · Deployment & URL capture (15 min) · Step 6 · Stretch goal (optional, ungraded)
Marks	The breakdown · Marking notes
Rules and support	Use of AI tools · Submission checklist

The assignment at a glance

What you're building

You will deploy a .NET 10 Web API to Azure App Service and connect it to a cloud database. You can use an existing API solution from your previous labs, or you can build upon the baseline provided in SDBP 031 Assignment 01.

Your deployed API must expose **two public GET** operations that retrieve data from your cloud database, and your source code must be hosted in a **public Git repository**.

Important: Everything in this addendum must run on free-tier services. Check the pricing tier on every resource before you create it.

What you'll have learned

- How to provision and manage cloud infrastructure using Microsoft Azure.
- How to securely connect a web application to a cloud database without hardcoding secrets in source control.
- How to expose a locally developed .NET application to the public internet via Azure App Service.
- Standard industry version control practices using Git, GitHub, or Azure DevOps.

The one idea behind the setup

Zero-cost cloud infrastructure. You are strictly required to use the free tier for all services. This deployment should cost you nothing, demonstrating how to sandbox and test cloud-native applications responsibly.

Step by step

Step 1 · Source Control (about 15 minutes)

Initialize a Git repository for your .NET solution. Create a remote repository on GitHub or Azure DevOps and push your code.

- The repository must be public so it can be evaluated.
- You must demonstrate standard version control practices by having at least two distinct commits in your commit history.

Step 2 · API requirements (about 15 minutes)

Ensure your .NET API project contains at least two functional GET endpoints that interact with a database. You can start fresh or use one of the projects that we used in our class as long as it can demonstrate database connectivity.

Step 3 · Provision the cloud database (about 20 minutes)

Create a cloud database (such as Azure SQL Database) in your Azure environment. **You must strictly select the free tier options** — for example, the Azure SQL Database Free Offer — to ensure nothing costs you.

Step 4 · App Service configuration (about 30 minutes)

Deploy your API to Azure App Service.

- Select the Free (F1) pricing plan.
- Navigate to your App Service settings and configure your environment variables. You must add your database connection string here rather than keeping it in your appsettings.json.
- Ensure that public access is enabled on the App Service so the endpoints can be reached externally.

Step 5 · Deployment & URL capture (about 15 minutes)

Test your two GET endpoints in the browser or via curl once deployed.

- Create a text file named **endpoints.txt** and paste the two working public API URLs inside it.
- Create a second text file named **url.txt** and paste the public link to your GitHub or Azure DevOps repository.
- Capture the required screenshots outlined in the submission checklist. Use the naming conventions used.

Step 6 · Stretch goal (optional and ungraded)

In enterprise scenarios, secrets are heavily protected. For an additional challenge, which will not be graded, put the database connection string key in an Azure Key Vault and allow the App Service to access it through a system-assigned managed identity.

Marks

The breakdown

Sr No	Description	How it's checked	Marks
1	Functional API endpoints	endpoints.txt containing two valid, accessible GET endpoints (2 marks each)	4
2	Source control	url.txt containing a valid, public GitHub or Azure DevOps repository URL demonstrating at least 2 commits	3
3	App Service provisioning (3.jpeg)	Screenshot of the Azure App Service hosted in your environment	1
4	Cloud database provisioning (4.jpeg)	Screenshot of the database running in your cloud environment	1
5	Configuration and security (5.jpeg)	Screenshot of the App Service environment variable successfully connecting to the database	1
	Total		10

Marking notes

- **A submission with broken URLs, private repositories, or inaccessible endpoints will lose the associated functional marks.**
- Ensure your App Service and database remain active, and on the free tier, until the assignment grades are published.

Rules and support

Use of AI tools

You are permitted to use AI tools to help understand Azure portal navigation, interpret deployment errors, or generate Azure CLI commands.

However, the work you hand in must be written by you. You may be asked to walk through any part of your configuration and explain why it works.

Submission checklist

- ☐ endpoints.txt containing the two public GET endpoint URLs
- ☐ url.txt containing the public GitHub or Azure DevOps repository URL
- ☐ Screenshot 1: Azure App Service hosted in your environment
- ☐ Screenshot 2: the deployed cloud database
- ☐ Screenshot 3: App Service environment variables showing the database connection
- ☐ your name is included in the submission, and your ZIP file is named per the Submission Procedure above

Important: Never commit or submit a connection string or API key to your public repository. Keep secrets in App Service environment variables, user-secrets, or Key Vault.